SmartFall: Watch-based Application

Introduction: SmartFall System 2
- Architecture Overview 2
- SmartFall Ecosystem 3
- Configuration 5
- Cloud Environment Configuration (PHP Tunnel) 6
- Example Setup (Default) 7

Installation and Setup 7
- Clone the Repository 7
- Enable Developer Options (First-Time Setup) 8
- Open the Project in Android Studio 8

Daily Workflow: Starting the App on Phone and Watch 8
- Navigation and User Interaction 10
  - HELP IS ON THE WAY Screen 11
  - Checking Battery 11

Usage of Smartwatch with SmartFall 11
- How to Force Quit or Shut Down the WearOS App on the Smartwatch 12

Retrieve the Sensed Data from Couchbase 12

How to Set Up Couchbase on a Local Computer 13

Department of Computer Science
Texas State University

© 2025 SmartFall Project. All rights reserved.

Introduction: SmartFall System

The following sections provide a comprehensive overview of the **SmartFall System**, detailing its architecture, ecosystem components, and configurable parameters. You'll gain insight into how the system is structured, how data flows between the watch, phone, and cloud components, and how various configuration settings influence its behavior. This information is essential for developers, researchers, or engineers looking to understand, deploy, or extend the SmartFall platform.

Architecture Overview

As illustrated in Figure 01, the **SmartFall system** follows a modular, event-driven architecture that integrates mobile-wearable communication, real-time sensor processing, machine learning inference, and cloud-synced data storage.

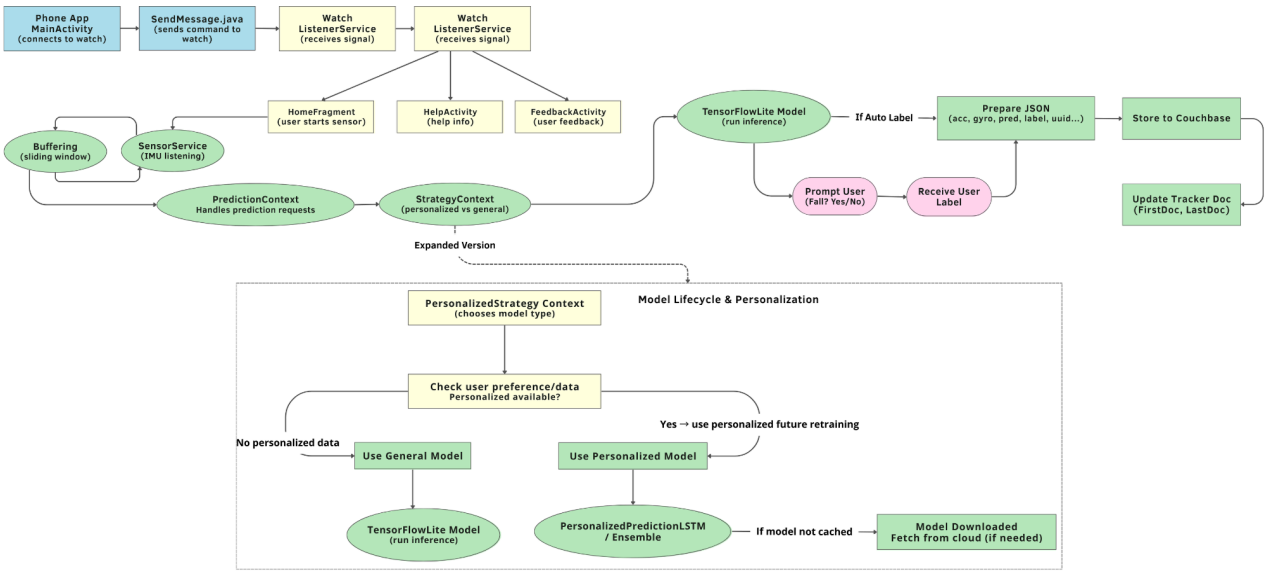


Figure 01: Flowchart of the system architecture

Phone-to-Watch Activation

- The **Phone App** acts as a lightweight controller. It initiates communication with the **Watch App** using SendMessage.java.
- The **Watch ListenerService** receives the signal and launches the main activity, initializing the user interface and background services.

Sensor Data Collection and UI Flow

- From the **HomeFragment**, the user can start or stop IMU data collection.
- The **SensorService** continuously listens to accelerometer and gyroscope streams.
- Data is buffered into sliding windows (e.g., 128-sample segments) for inference.

Model Prediction Pipeline

- The buffered IMU window is passed to PredictionContext.java, which routes the data through the appropriate model.
- StrategyContext determines whether to use a **general** or **personalized** model based on configuration and user availability.

- The selected model (e.g., TensorFlowLiteLSTM or PersonalizedPredictionLSTM) performs fall classification on-device.

User Label Prompting

- If the prediction confidence is below the threshold or manual confirmation is enabled, a **prompt is shown to the user** (e.g., “Was this a fall?”).
- The user's input is captured and used to **label the data** for storage and future personalization.

Data Packaging and Storage

- The result (sensor data + prediction + label + metadata) is formatted as a JSON object.

```
{
"uuid": "user123",
"timestamp":
"2025-06-20T14:33:00Z",
"acc": [...],
"gyro": [...],
"prediction_label": "fall",
"confidence": 0.91,
"model_used": "PERSONALIZED_LSTM",
"type": "FP"
}
```

- The labeled instance is stored in **Couchbase**, both locally and optionally synced to a remote bucket.
- A **tracker document** is updated to maintain session boundaries (e.g., firstDoc, lastDoc).

Personalization Lifecycle

- Labeled data can be used in future sessions to fine-tune or switch to personalized models.
- The model downloader fetches updated model weights if not already cached on-device.

SmartFall Ecosystem

As illustrated in Figure 02 below, the SmartFall system follows an edge-to-cloud architecture where the smartwatch serves as the primary device for sensing, inference, and labeling, while the smartphone and cloud components provide support for configuration, storage, and personalization.

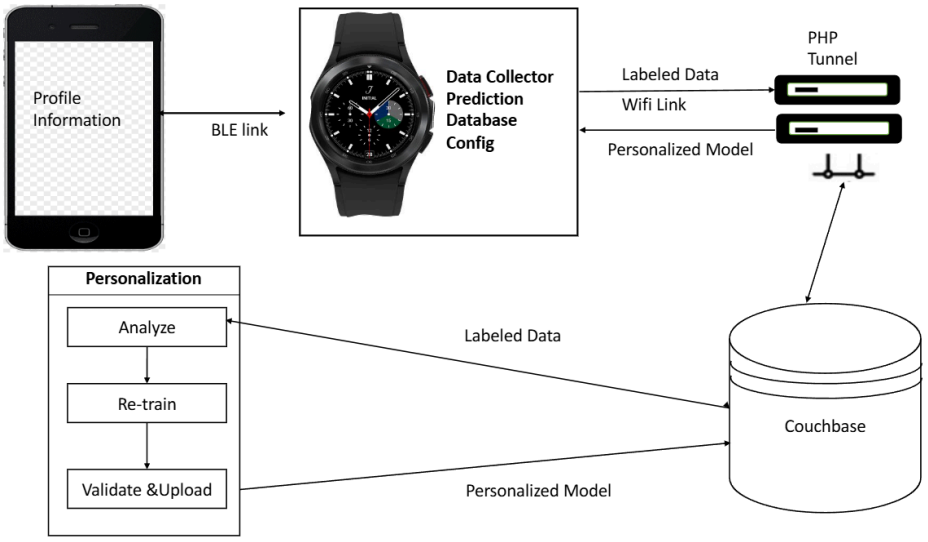


Figure 02: High-level architecture of the SmartFall system, showing data flow between the phone, watch, cloud database, and model personalization pipeline.

1. Smartphone Configuration
    a. The smartphone app is used to set up the user's profile, including personal ID, preferences, and model configuration (e.g., general vs. personalized).
    b. Profile settings are transmitted to the smartwatch over BLE (Bluetooth Low Energy) and stored locally on the watch for runtime reference.
2. Real-Time Data Collection on Watch
    a. The smartwatch continuously collects IMU data (accelerometer and gyroscope) through its background SensorService.
    b. This raw sensor data is buffered into fixed-size sliding windows and fed into the model for activity analysis.
3. On-Device Processing and Inference
    a. A lightweight TensorFlow Lite model (general or personalized) is deployed and executed locally on the watch.
    b. Fall detection is performed in real-time, and the confidence score is evaluated against a configurable threshold.
    c. If confidence is high, the fall label is accepted.
    d. If confidence is low, the user is prompted: "Did you fall?"
4. User Feedback and Labeling
    a. The user’s response helps generate labeled data, which includes:
        i. Raw IMU window
        ii. Model prediction
        iii. User label
        iv. Timestamp, UUID, model type
5. Cloud Integration and Personalization
    a. Labeled data is sent to Couchbase via a Wi-Fi-based PHP tunnel for archival and training.
    b. A cloud-side personalization module periodically:
        i. Analyzes labeled data
        ii. Retrains personalized models
        iii. Validates and uploads updated models
6. Model Delivery
    a. The smartwatch fetches personalized models from the cloud if a valid update is available.

b. The updated model is used in subsequent inference loops, improving accuracy over time through continual adaptation.

Configuration

The **Watch app** is fully configurable via the **SmartFallConfig.java** file (*wear > java > config > SmartFallConfig.java*). It controls model selection, personalization strategy, runtime behaviour, and remote database settings.

Configuration Parameters:

Parameter	Possible Values	Description
MODEL_TYPE	"PERSONALIZED", "LSTM", "DEFAULT"	Determines the overall model strategy: • PERSONALIZED uses per-user adaptation • LSTM uses general pre-trained model • DEFAULT (fallback/backup logic)
PERSONALIZED_STRATEGY	"LSTM", "ENSEMBLE"	Specifies which personalized model architecture to use: • LSTM = single LSTM model • ENSEMBLE = multiple models with majority voting
MODEL_FROM	"OFFLINE", "ONLINE"	Determines whether models are loaded: • OFFLINE = from local assets folder • ONLINE = downloaded dynamically
OFFLINE_MODEL_FILE	"Saved_model128.tflite"	Name of the local *.tflite* file bundled with the app (used if MODEL_FROM = OFFLINE)
OFFLINE_MODEL_THRESHOLD	0.0–1.0 (e.g., 0.9f)	Minimum confidence score required for automatic labeling. After a model makes a prediction, if the confidence is lower than this threshold, the system will prompt the user with “Did you fall?” instead of trusting the model.
REMOTE_BUCKET_NAME	"smart-fall-data"	Target Couchbase bucket where labelled documents are uploaded
API_KEY	"demoKey!xyz9&'1234 ABC" (or others)	Authentication key for syncing with the remote Couchbase Sync Gateway
ENV_FLAG	"SMART_FALL", "GANYMEDE"	Environment selector for remote deployment (used to toggle server configs)
ALPHA_LIMIT	Integer (e.g., 20)	Length of the **alpha queue** (may represent smoothed output memory or state history)
BETA_LIMIT	Integer (e.g., 128)	Size of the input **window** for model inference (i.e., number of IMU samples per prediction)
STEP_SIZE	Integer (e.g., 10)	Step size for sliding window advance

Cloud Environment Configuration (PHP Tunnel)

The SmartFall system uses environment-specific PHP endpoints (as found in *wear > java > config > SmartFallConfig.java*) to communicate with Couchbase through a secure web-based tunnel. These endpoints are defined in the getCloudConfig() method of the configuration class.

```
public static CloudIPConfig getCloudConfig() {
        CloudIPConfig urlPaths = new CloudIPConfig();
        if (ENV_FLAG == "MERCURY") {
            …
        }
        else if (ENV_FLAG == "GANYMEDE") {
           …
        }
        else if (ENV_FLAG == "EUROPA") {
           …
        }else if (ENV_FLAG == "SMART_FALL") {
            System.out.println("It is smartfall server URL Path");
            urlPaths.checkVersion =
"https://server-name.edu/smartFall/checkversion.php";
            urlPaths.downloadModel =
"https://server-name.edu/smartFall/downloadmodels.php";
            urlPaths.dataUpload =
"https://server-name.edu/smartFall/upsertdocuments.php";
        }

        return urlPaths;
    }
```

Depending on the selected environment (ENV_FLAG), the application dynamically sets the appropriate URLs for the following operations:

- checkVersion – checks if a newer personalized model is available for download.
- downloadModel – retrieves the current machine learning model from the server.
- dataUpload – uploads labeled sensor data to Couchbase via PHP.

Supported environments include:

Environment	**Server URL Base**	**Description**
MERCURY	http://server-name.edu/couchbase-prod/	Production environment
GANYMEDE	http://server-name.edu/	Alternate dev/test server
EUROPA	http://server-name.edu/	Development environment

SMART_FALL	https://server-name.edu/smartFall/	Official SmartFall production server

Each server hosts three PHP endpoints:

- checkversion.php
- downloadmodels.php
- upsertdocuments.php

These PHP scripts act as a tunnel between the Android watch app and Couchbase, handling version checks, data transfers, and model downloads securely over HTTP/HTTPS.

> To change the server used by the app, simply update the ENV_FLAG in SmartFallConfig.java.

Example Setup (Default)

```
public static final String MODEL_TYPE = "PERSONALIZED";
public static final String PERSONALIZED_STRATEGY = "LSTM";
public static final String MODEL_FROM = "OFFLINE";
public static final String OFFLINE_MODEL_FILE = "saved_model128.tflite";
public static final float OFFLINE_MODEL_THRESHOLD = 0.9f;
public static final String REMOTE_BUCKET_NAME = "smart-fall-data";
public static final String ENV_FLAG = "SMART_FALL";
public static final int ALPHA_LIMIT = 20;
public static final int BETA_LIMIT = 128;
```

Installation and Setup

To successfully build and run the SmartFall application, please ensure it is correctly installed and configured on your development environment using the steps outlined below.

Clone the Repository

1. Create a folder on your computer (e.g., `SmartFall`).

```
mkdir SmartFall
```

2. Open the folder in your terminal.

```
cd SmartFall
```

3. Run the following commands:

```
git clone https://github.com/txst-cs-smartfall/SmartFall-Multi-Model-Watch.git
cd SmartFall-Multi-Model-Watch
```

```
git checkout main
```

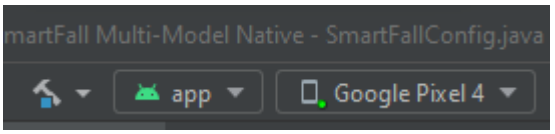
Enable Developer Options (First-Time Setup)

Perform these steps **on both your Android phone and your WearOS watch**:

1. Open **Settings > About**.
2. Tap **Build Number** repeatedly to enable Developer Mode.
3. Navigate to **Settings > Developer Options**.
4. Enable **ADB Debugging**.

Open the Project in Android Studio

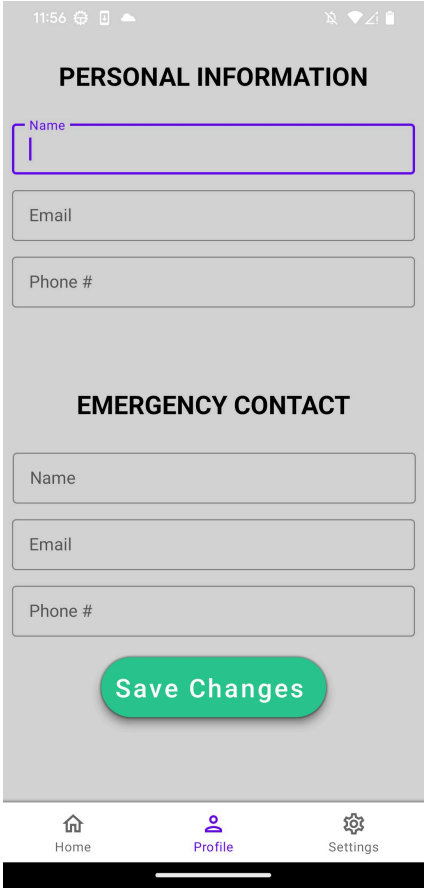
1. Launch **Android Studio (Giraffe or later)**.
2. Choose **File > Open** and select the SmartFall-Multi-Model-Watch directory.
3. Allow Gradle to sync and build the project (this may take a few minutes).
4. If the build doesn't start automatically, click **Build > Make Project** from the top toolbar, as shown below:



> You can now run the phone and watch apps on connected devices using the device manager or ADB of Android Studio.

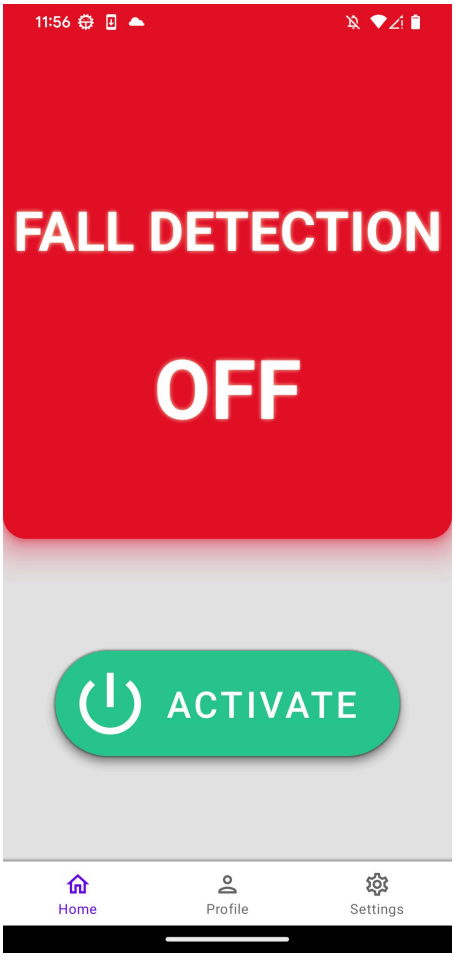
Daily Workflow: Starting the App on Phone and Watch

1) Find the SmartFall App on your phone and launch it. You will see the following screen when it is launched successfully:



2) Launch the WearOS app on the watch, and the following “Fell” screen is displayed (Profile only needs to be created once on the phone):

3) After creating the profile, you will see the home screen with the “Activate” button. By pressing this button, you can start data transmission in the application.



4) The watch screen will remain the same while creating the profile and activating the application.

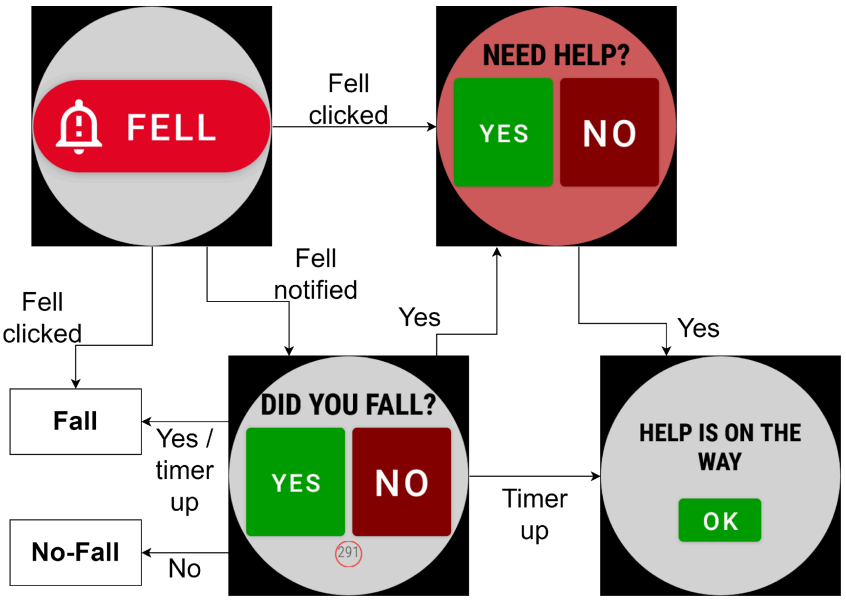
5) Once activated, we will see numbers flashing on the watch screen, which means the application has been installed, connected with the phone, and activated successfully. Now you can start testing.

Navigation and User Interaction

The flowchart below illustrates the navigation and user interaction flow within the SmartFall application. While the App is running, whenever the App predicts that you might be falling, the "DID YOU FALL?" prompt/notification will be displayed. The watch will ring a tone and vibrate to remind you about the prompt/notification.

We have two options here:

1) When you respond with "YES", it will ask you to respond to the second question, i.e., "DID YOU FALL?" Respond with "NO" if you did not fall (you will respond this way almost all the time).
2) When you respond with "YES", it will display the "HELP IS ON THE WAY" screen. This will send an email to your caregiver.



HELP IS ON THE WAY Screen

If you did not respond to the yes/no notification screen, 5 minutes later it will automatically display the “HELP IS ON THE WAY” screen. If “HELP IS ON THE WAY” shows up by mistake, swipe right to get back to the yes/no prompt (the previous screen) and respond to it.

1. Sometimes, you can get many prompts one after another, just respond one by one.
2. If at any point you press the “Deactivate” button on the phone, the system will stop collecting the data, and you will see the earlier screen “Activate” button. You can press the “Deactivate” button whenever you want to stop using the watch or when the battery is low. It is best to put the watch on the charger after you stop it.
3. If you press the “Activate” button, the system will go back to collecting data again.
4. The battery will typically last 5+ hours (Huawei watch) in the watch if you run the App continuously. To check the level of battery, go to the home screen by pressing the top button on the watch and swiping down.

Checking Battery

If you forgot to check the battery, when the battery is critically low, the watch will vibrate, and it will automatically power off. If that happens, just recharge it and then force quit the WearOS app when there is sufficient power to perform that operation.

Usage of Smartwatch with SmartFall

1. The way you interact with the watch is through the two side buttons and by touching or swiping the screen. The watch automatically switches to ambient mode after 5 seconds of inactivity or when you cover the screen with your palm.
2. Tapping the top button once brings you to the home screen. Tapping it a second time opens the list of applications. Pressing the top button repeatedly toggles between the home screen and the app list. Pressing and holding the top button activates the voice assistant; if this is triggered unintentionally, you can press the button again to return to the home screen. The bottom button is not used for SmartFall.
3. You can also interact using the touchscreen. Tap the screen to select or confirm an option. Swipe up to view the currently running app, swipe down to access the shortcut menu (e.g., to check battery status), and swipe from edge to edge to switch between watch faces. Swiping right takes you back to the previous screen.

4. If any other notifications appear by mistake, simply press the top button to return to the home screen. Press it again to open the list of apps and tap “WearOS” to return to the SmartFall application.

How to Force Quit or Shut Down the WearOS App on the Smartwatch

To manually close the WearOS app on your HUAWEI smartwatch, for example, follow these steps:

1. Press the top side button twice to open the app list.
2. Scroll to and select “Settings”.
3. Navigate to “Apps & notifications” and tap to open it.
4. Select “App info” to view the list of installed apps.
5. Locate and tap “WearOS” in the app list.
6. Tap “Force stop”, then confirm by pressing the check mark.

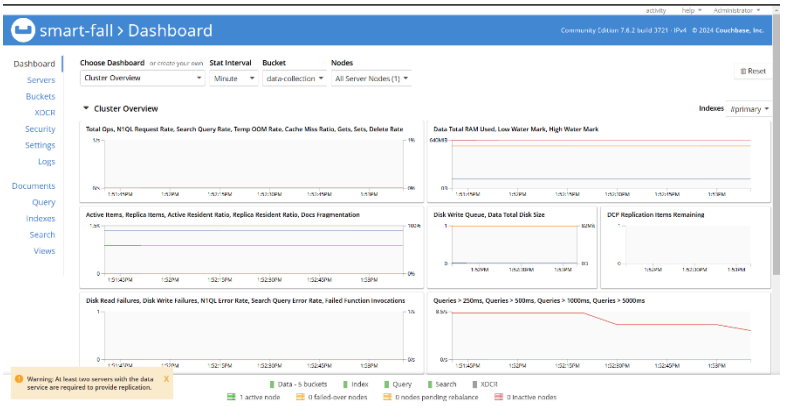
The WearOS app will now be shut down completely on the watch.

Retrieve the Sensed Data from Couchbase

Since this system integrates with Couchbase, the following section demonstrates how to retrieve data using the Couchbase Web Console. You’ll need valid login credentials if you have an active server and account. If not, you may modify the app to connect with your own dataset or database backend.

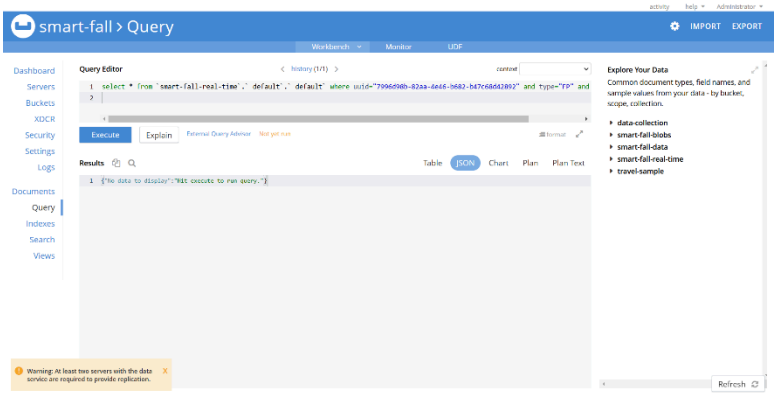
1. The data can be downloaded from the Couchbase server by accessing it through the provided URL.
2. Log in using your registered administrator username and password.

3. Once signed in, you will see the main dashboard. From the left sidebar, click on “Query” to access the query interface.

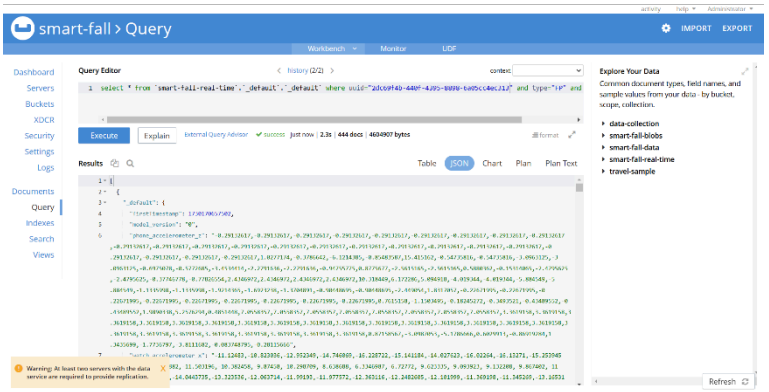


4. Sample Query to find data: (Change the UUID and TYPE accordingly). After writing the query, click **Execute**.

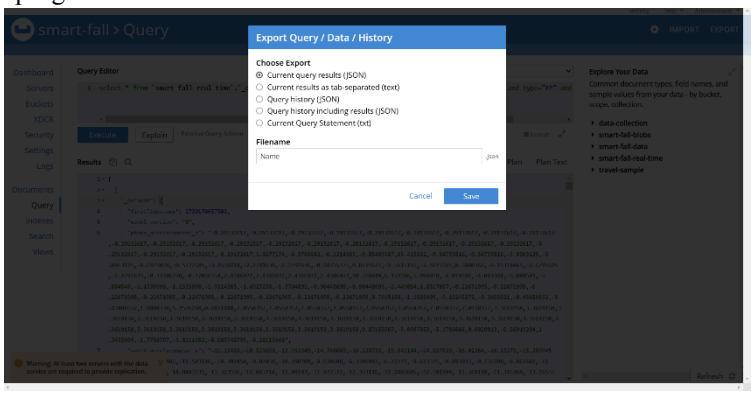
```
select * from `smart-fall-real-time`.`_default`.`_default` where
uuid="7996d98b-82aa-4e46-b682-b47c68d42892" and type="FP" and
type!="tracker"
```



5. You will see if the data has been recorded.



6. Click **Export** in the top right corner.



7. Select the export option as JSON and name the file. Then click on the **Save** button. The output JSON file will be saved on your local computer.

How to Set Up Couchbase on a Local Computer

This section outlines the steps to set up a local Couchbase Server instance and integrate it with the SmartFall system. It covers the creation of data buckets for storing sensor and cognitive test data, configuring the PHP SDK for backend access, and enabling communication between the Couchbase database and the React frontend. This setup allows for efficient data logging, retrieval, and visualization within the SmartFall application.

1. Install Couchbase Server Locally

Step 1.1: Download and Install Couchbase

1. Visit: `https://www.couchbase.com/downloads`

2. Select: Couchbase Server (Community Edition)

3. Choose your operating system (Windows, macOS, Ubuntu)

4. Download and install the Couchbase Server

Step 1.2: Start Couchbase Web UI

- Open a browser and go to: `http://localhost:8091`

- Follow the setup wizard:

  • Create an admin username and password (e.g., Administrator, password123)

  • Accept the default services: Data, Query, Index

  • Initialize the cluster

2. Create Buckets for SmartFall

You will need the following buckets:

- smart-fall-data: to store movement sensor data

Step 2.1: Create Buckets

1. Go to Buckets > Add Bucket

2. Name the bucket (e.g., smart-fall-data)

3. Set RAM quota (e.g., 100MB is sufficient)

3. Install and Configure PHP Couchbase SDK

Step 3.1: Install SDK with Composer

1. Ensure Composer is installed: https://getcomposer.org/

2. In your project directory (where getscores.php is located), run:

```
composer require couchbase/couchbase
```

Step 3.2: Enable the Couchbase PHP Extension (ext-couchbase)

This extension is necessary for PHP to communicate with Couchbase.

<u>On Ubuntu/Linux:</u>

```
sudo apt install php-couchbase
sudo systemctl restart apache2
```

<u>On macOS (with Homebrew):</u>

```
brew install couchbase/php-couchbase
```

Add the following to `php.ini`:

```
extension=couchbase.so
Check php.ini location:
php --ini
```

<u>On Windows (XAMPP/WAMP):</u>

1. Download php_couchbase.dll
2. Place in C:\xampp\php\ext
3. Edit php.ini:

   extension=php_couchbase.dll

4. Restart Apache

Step 3.3: Verify the Installation

Create a test PHP file:

```
<?php
      phpinfo();
?>
```

Load it in the browser. If you see a couchbase section, the extension is active.

4. Connect and Query Couchbase in PHP

Simple connection example:

```
use Couchbase\Cluster;
use Couchbase\ClusterOptions;

$options = new ClusterOptions();
$options->credentials("Administrator", "your-password");

$cluster = new Cluster("couchbase://127.0.0.1", $options);
$bucket = $cluster->bucket("smart-fall-data");
```

5. Sample Data Format

Movement Document (smart-fall-data):

```
{
  "uuid": "user-1234",
  "type": "TP",
  "firstTimestamp": 1748822400000,
  "watch_accelerometer_x": "0.12,0.18,...",
  "watch_accelerometer_y": "-0.05,0.01,...",
  "watch_accelerometer_z": "0.98,1.01,..."
}
```

6. Accessing Data from the React Frontend

1. React sends a POST request to getscores.php
2. Request includes uuid, bucket, and time window
3. PHP queries Couchbase and returns JSON
4. React renders charts using the response

Ensure the Couchbase URL in PHP is:

```
$cluster = new Cluster("couchbase://127.0.0.1", $options);
```